

CENG454 HW3 - Core Breach

Full Name: Orhan Ciftci
Student ID: 220444403
GitHub Repository URL: [https://github.com/\[username\]/CENG454-HW3-Orhan-Ciftci-220444403](https://github.com/[username]/CENG454-HW3-Orhan-Ciftci-220444403)
Homework Title: Core Breach: Pattern-Driven Systems Prototype
Submission Date: 17 May 2026

Important: Replace the GitHub username/date if needed, then add real screenshots from Unity and GitHub before final submission.

2. Gameplay Overview

Core Breach is a short 2D defend-the-core prototype. The player controls a facility defender, moves around the arena, aims with the mouse, and fires at hostile units before they reach the central energy core. The player wins when all spawned enemies are destroyed and the core survives. The player loses when enemy contact damage reduces the core health to zero.

3. Architecture Overview

CoreHealth owns objective health and publishes health/destruction events. GameHud, CoreDamageFlash, and GameStateController subscribe to core events without being directly controlled by the core. PlayerWeapon owns firing input but delegates firing behavior to WeaponStrategyBase assets. ProjectilePool owns reusable projectile instances, while each Projectile resets velocity, lifetime, damage, and owner state on reuse. WaveSpawner owns encounter pacing and alternates between direct and strafing enemy prefabs. EnemyUnit owns enemy health/contact damage and delegates movement to IEnemyMovementStrategy.

Architecture flow: PlayerWeapon -> WeaponStrategyBase -> ProjectilePool -> Projectile -> IDamageable. EnemyUnit -> IEnemyMovementStrategy. WaveSpawner -> EnemyUnit. CoreHealth events notify GameHud, CoreDamageFlash, and GameStateController. WaveSpawner wave completion also notifies GameStateController.

4. Pattern Justification Table

Pattern / Technique	Classes or Interfaces Involved	Problem Solved	Why Appropriate
Interfaces	IDamageable, ITargetProvider, IWeaponStrategy, IEnemyMovementStrategy, IPoolable, IDamageable	Enables systems from depending on concrete classes; defines behavior contracts	Damageable; enemies use any movement strategy
Observer	CoreHealth events, WaveSpawner, GameHud, CoreDamageFlash, GameStateController	Enables loosely coupled dependencies; notifies interested parties	CoreHealth notifies HUD, visual feedback, or game-state
Strategy	WeaponStrategyBase, SingleShotWeaponStrategy, SpreadShotWeaponStrategy, StrafingWeaponStrategy, DirectWeaponStrategy	Encapsulates behavior; allows runtime swapping	WeaponStrategyBase defines movement strategy
Object Pool	ProjectilePool, Projectile, IPoolable	Projectiles are spawned repeatedly; reuse objects	Reusing projectiles avoids instantiate/destroy churn and reset logic
Decorator	DamageBoostWeaponDecorator, WeaponStrategyBase, ExistingWeaponStrategy	WeaponStrategyBase wraps existing strategies	WeaponStrategyBase wraps existing strategies

5. Screenshots and Evidence

Screenshot 1: Scene or Game View showing the playable arena and defendable objective.

Screenshot 2: Evidence of IDamageable plus relevant implementations/users.

Screenshot 3: Evidence of Observer-style event flow: CoreHealth event and subscribers.

Screenshot 4: Evidence of Strategy setup: weapon strategies or enemy movement strategies.

Screenshot 5: Runtime or Inspector evidence of Projectile Pool.

Screenshot 6: Evidence of DamageBoostWeaponDecorator wrapping a weapon strategy.

Screenshot 7: Repository evidence of feature branches, pull requests, or merge flow.

6. Debugging Story

During testing, one bug I encountered was that projectiles could keep their old movement state when reused. I diagnosed it by watching pooled projectiles after they were returned and fired again; some shots did not start cleanly from the muzzle direction. The problem was that the object pool reused the same Rigidbody2D object, but the projectile needed to clear velocity and lifetime-related state each time it was taken from or returned to the pool. I fixed it by resetting remainingLifetime, damage, ownerLayer, linearVelocity, and angularVelocity inside OnTakenFromPool and OnReturnedToPool.

7. Troubleshooting Answer

The visible symptom would be duplicated reactions after the core-damaged event, such as the same inactive or reused enemy playing a sound, spawning VFX, or running damage-response logic more than once. The root cause is that the enemy subscribed to the event during one lifetime but did not unsubscribe when it died or returned to the pool. Since pooled objects are not destroyed, the old subscription remains in the publisher invocation list and becomes a ghost subscriber. The exact fix is to unsubscribe in OnDisable, OnReturnedToPool, or the enemy death cleanup path, matching the place where the subscription was created. A prevention rule is that every event subscription must have a clearly paired unsubscribe in the same lifecycle ownership model, and pooled objects must reset both gameplay state and external subscriptions before reuse.

8. Retrospective

The most helpful design choice was using interfaces and events around the core loop because it kept the objective, HUD, visual feedback, and game-state logic separated. Strategy also made the prototype easier to expand because new weapons and movement styles can be added without rewriting the player or enemy owner classes. If I continued the project, I would improve the encounter system with more wave profiles and add a small upgrade selection screen that creates more decorator combinations.